

Министерство науки и высшего образования Российской Федерации
Муромский институт (филиал)
федерального государственного бюджетного образовательного учреждения
высшего образования
**«Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»**
(МИ ВлГУ)

Факультет информационных технологий радиоэлектроники
Кафедра информационных систем

КУРСОВАЯ РАБОТА

по курсу Управление данными
на тему: Разработка программы для визуализации работы алгоритмов
сортировки

Руководитель

(уч. степень, звание)

Метелкин А. С.

(фамилия, инициалы)

(подпись)

(дата)

Студент ИС-123

(группа)

Смакотина Д. В.

(фамилия, инициалы)

(подпись)

(дата)

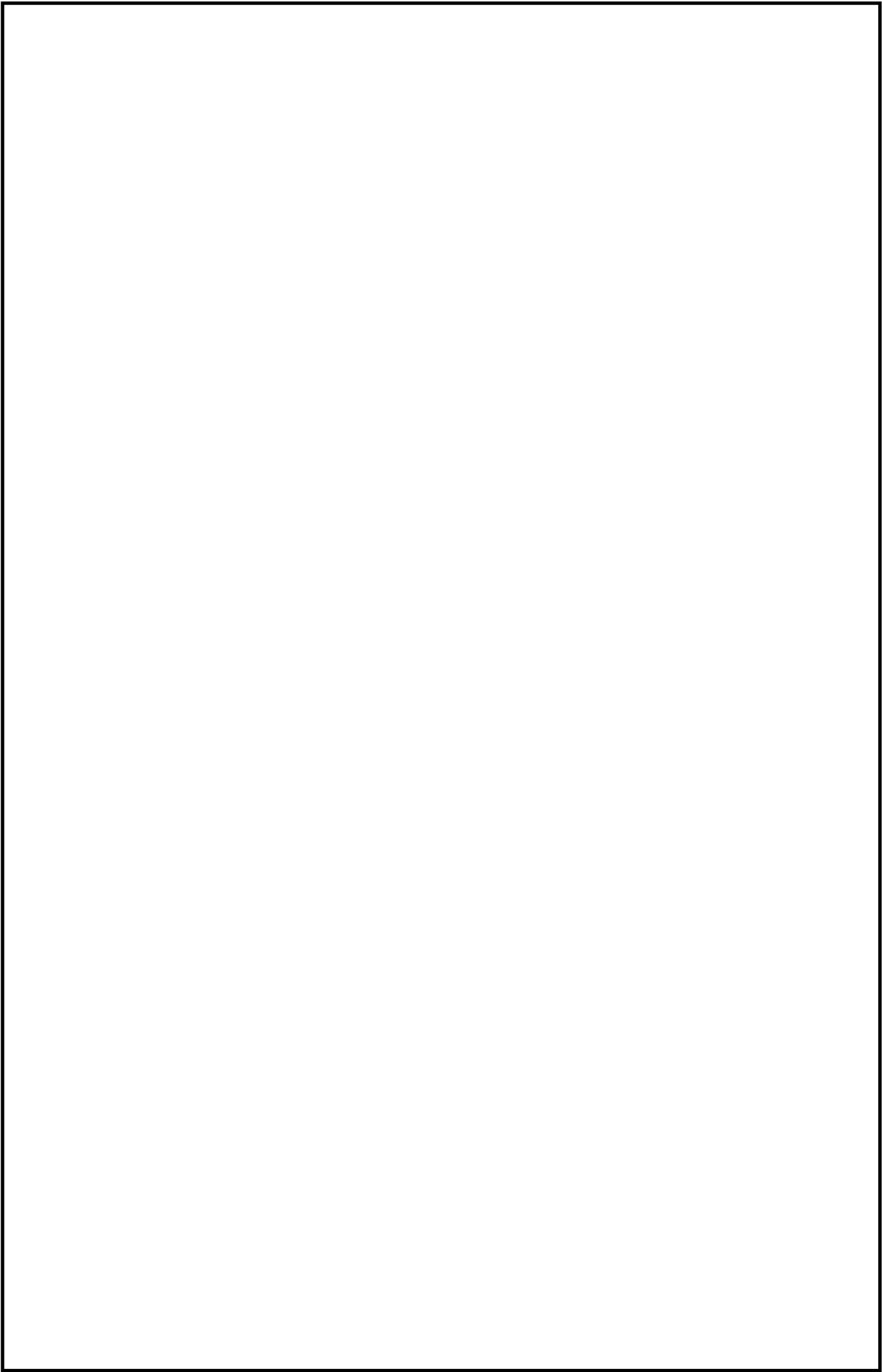
Члены комиссии

(подпись)

(Ф.И.О.)

(подпись)

(Ф.И.О.)



В курсовой работе отражена разработка приложения для визуализации алгоритмов сортировки, реализована обработка данных с использованием файлов различных форматов (TXT, CSV, JSON). Спроектирована архитектура программы на основе принципов объектно-ориентированного программирования с применением паттернов проектирования. Создан интерфейс, позволяющий загружать данные, выбирать алгоритм сортировки, наблюдать процесс выполнения и сохранять результат в файл. Реализованы алгоритмы сортировки: пузырьковая, быстрая и сортировка выбором, а также различные способы визуализации: консольная, JavaFX и Swing. Проведено тестирование основных модулей. Разработанное приложение обеспечивает наглядное представление работы алгоритмов и удобное взаимодействие пользователя.

Содержание

Введение 4

1. Анализ технического задания 6

2. Проектирование программы 8

3. Разработка программы..... 12

4. Тестирование 23

Заключение 28

Библиография 30

					МИВУ.09.03.02 020 ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата	Разработка программы для визуализации работы алгоритмов сортировки	Лит.	Лист	Листов
Разраб.		Смакотина Д. В.						
Провер.		Метелкин А.С.					4	
Реценз.						ИС-123		
Н. Контр.								
Утверд.								

Введение

В современных условиях изучение алгоритмов и структур данных является важной частью подготовки специалистов в области программирования. Одной из ключевых задач является понимание принципов работы алгоритмов сортировки, которые широко применяются при обработке данных. Для более наглядного изучения и анализа таких алгоритмов используются программные средства визуализации, позволяющие наблюдать процесс их выполнения в реальном времени. Подобные решения способствуют лучшему усвоению материала и повышают эффективность обучения.

Целью данного курсового проекта является разработка программного приложения для визуализации алгоритмов сортировки. Программа должна обеспечивать загрузку данных из файлов, выбор алгоритма сортировки, отображение процесса выполнения с использованием различных способов визуализации, а также сохранение результата в файл. Особое внимание уделяется построению архитектуры приложения с использованием интерфейсов и паттернов проектирования, обеспечивающих гибкость и расширяемость системы.

Для достижения поставленной цели в рамках проекта необходимо решить следующие задачи:

- Разработать архитектуру приложения с использованием интерфейсов для разделения логики программы;
- Реализовать алгоритмы сортировки (пузырьковая, быстрая, сортировка выбором);
- Обеспечить возможность выбора алгоритма, способа хранения данных и варианта визуализации;
- Реализовать загрузку и сохранение данных в различных форматах (ТХТ, CSV, JSON)
- Разработать несколько способов визуализации;
- Обеспечить возможность расширения программы для добавления новых алгоритмов и модулей.

					МИВУ.09.03.02 020 ПЗ	Лист
						4
Изм.	Лист	№ докум.	Подпись	Дата		

Результатом работы является программное приложение, реализующее визуализацию алгоритмов сортировки с возможностью выбора алгоритма, формата данных и способа отображения. Разработанная система демонстрирует применение паттернов проектирования и принципов объектно-ориентированного программирования, обеспечивает наглядное представление процесса сортировки и может использоваться в учебных целях.

					МИВУ.09.03.02 020 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		

1. Анализ технического задания

Разработка приложения для визуализации алгоритмов сортировки требует учёта особенностей предметной области, связанной с обработкой и отображением данных, а также с демонстрацией работы алгоритмов в реальном времени. В образовательной и практической деятельности подобные программы используются для наглядного представления процессов сортировки, что позволяет лучше понять принципы их работы и сравнить эффективность различных алгоритмов. В связи с этим при проектировании приложения особое внимание уделяется разделению логики сортировки и визуализации, а также обеспечению гибкости и расширяемости системы.

Предметная область включает обработку числовых данных, представленных в виде массивов, а также их визуализацию различными способами. Исходные данные поступают из файлов различных форматов (ТХТ, CSV, JSON) и преобразуются во внутреннее представление в виде массива целых чисел. В процессе выполнения алгоритмов происходит последовательное изменение элементов массива, которое должно корректно отображаться средствами визуализации без влияния на саму логику сортировки.

Анализ требований показал необходимость реализации модульной архитектуры с использованием интерфейсов, обеспечивающих взаимодействие между компонентами программы. Это позволяет разделить систему на независимые части: алгоритмы сортировки, модули загрузки и сохранения данных, а также модули визуализации. Такой подход упрощает сопровождение программы и позволяет расширять её функциональность без изменения существующего кода.

Важным требованием является использование паттернов проектирования. В рамках проекта применяются паттерны Factory для создания объектов (алгоритмов, визуализации и работы с данными), Strategy для выбора алгоритма сортировки, а также Observer для передачи изменений состояния массива в

					МИВУ.09.03.02 020 ПЗ	Лист
						6
Изм.	Лист	№ докум.	Подпись	Дата		

модуль визуализации. Использование данных паттернов обеспечивает гибкость архитектуры и независимость компонентов системы.

Также в техническом задании предусмотрена возможность выбора алгоритма сортировки, способа хранения данных и варианта визуализации. Это реализуется за счёт использования фабрик и интерфейсов, позволяющих динамически определять необходимые компоненты во время выполнения программы. Такой подход делает систему универсальной и адаптируемой под различные сценарии использования.

Особое внимание уделено требованию расширяемости. Архитектура программы спроектирована таким образом, чтобы добавление новых алгоритмов сортировки, форматов данных или способов визуализации не требовало изменения уже реализованных модулей. Для этого достаточно создать новый класс, реализующий соответствующий интерфейс, и зарегистрировать его в фабрике.

В результате анализа технического задания определены основные компоненты системы, их взаимодействие и принципы построения архитектуры. Выбранный подход, основанный на использовании интерфейсов и паттернов проектирования, полностью соответствует поставленным требованиям и обеспечивает гибкость, расширяемость и наглядность работы приложения.

					МИВУ.09.03.02 020 ПЗ	Лист
						7
Изм.	Лист	№ докум.	Подпись	Дата		

2. Проектирование программы

Для разработки программы визуализации алгоритмов сортировки сначала была определена основная задача проекта. Программа должна была загружать данные из файла, выполнять сортировку массива в оперативной памяти и отображать процесс сортировки различными способами визуализации. Дополнительно требовалось обеспечить возможность выбора алгоритма сортировки, формата файлов и типа визуализации. Также важным требованием была расширяемость программы, чтобы в дальнейшем можно было добавлять новые алгоритмы и способы отображения без изменения уже существующего кода.

После определения основных требований была спроектирована общая структура программы. Для упрощения разработки и поддержки проекта было принято решение разделить систему на несколько независимых модулей, каждый из которых отвечает только за свою задачу. В результате программа была разделена на модули алгоритмов сортировки, визуализации, работы с файлами, хранения данных и создания объектов. Такое разделение позволило сделать архитектуру более понятной и уменьшить зависимость компонентов друг от друга.

Следующим этапом стало проектирование подсистемы алгоритмов сортировки. Поскольку программа должна поддерживать несколько различных алгоритмов, было важно организовать их единым образом. Для этого был создан интерфейс `SortAlgorithm`, который определяет общий метод сортировки массива. Благодаря интерфейсу все алгоритмы имеют одинаковую структуру взаимодействия с остальной программой. После создания интерфейса были реализованы конкретные алгоритмы сортировки: пузырьковая сортировка, быстрая сортировка и сортировка выбором. Каждый алгоритм работает независимо от визуализации и выполняет изменения только внутри массива данных, находящегося в оперативной памяти.

После проектирования алгоритмов была разработана система визуализации.

					МИВУ.09.03.02 020 ПЗ	Лист
						8
Изм.	Лист	№ докум.	Подпись	Дата		

Основной задачей на этом этапе стало отделение логики сортировки от отображения данных. Это было необходимо для того, чтобы алгоритмы могли работать одинаково независимо от используемой библиотеки интерфейса. Для решения этой задачи был создан интерфейс `SortObserver`. Через него алгоритмы передают информацию о текущем состоянии массива визуализатору. Таким образом, сортировка не зависит от конкретного способа отображения, а визуализация только получает обновлённые данные и отображает их пользователю.

На основе интерфейса визуализации были реализованы три различных варианта отображения: консольная визуализация, визуализация на основе `Swing` и визуализация с использованием `JavaFX`. Консольный вариант выводит массив в текстовом виде и используется как наиболее простой способ отображения процесса сортировки. `Swing` и `JavaFX` предоставляют графическую визуализацию в виде столбчатых диаграмм, где высота каждого столбца соответствует значению элемента массива. В `JavaFX` дополнительно были реализованы выбор файлов через графический интерфейс, настройка скорости анимации и подсветка сравниваемых элементов.

После этого была спроектирована подсистема работы с файлами. Поскольку программа должна поддерживать несколько форматов входных и выходных данных, было принято решение создать отдельные интерфейсы для загрузки и сохранения информации. Для загрузки был создан интерфейс `DataLoader`, а для сохранения — `DataSaver`. Затем были реализованы классы для работы с текстовыми файлами, CSV-файлами и JSON-файлами. Такой подход позволил унифицировать работу с данными и сделать поддержку новых форматов максимально простой.

Следующим важным этапом стало внедрение фабрик. Для создания объектов алгоритмов, визуализаторов и загрузчиков данных были реализованы специальные классы-фабрики: `AlgorithmFactory`, `VisualizerFactory`, `DataLoaderFactory` и `DataSaverFactory`. Использование фабрик позволило убрать из основного кода большое количество условных операторов и централизовать

создание объектов в отдельных классах. Теперь основной модуль программы не зависит от конкретных реализаций алгоритмов или визуализации и работает только через интерфейсы. Это значительно упрощает расширение программы и улучшает структуру проекта.

После завершения проектирования основных компонентов была реализована логика взаимодействия между ними. Пользователь выбирает входной файл, после чего соответствующий загрузчик считывает данные и преобразует их в массив. Далее через фабрику создаётся выбранный алгоритм сортировки и объект визуализации. Во время сортировки алгоритм изменяет массив в оперативной памяти и после каждого значимого шага вызывает обновление визуализации через интерфейс наблюдателя. После завершения сортировки результат сохраняется в файл выбранного формата.

При проектировании также особое внимание уделялось расширяемости программы. Благодаря использованию интерфейсов и фабрик добавление нового алгоритма или нового способа визуализации не требует изменения существующей логики системы. Для добавления нового алгоритма достаточно создать новый класс, реализовать интерфейс `SortAlgorithm` и зарегистрировать его в фабрике. Аналогичным образом можно добавлять новые форматы файлов или новые варианты отображения.

Таким образом, в основе архитектуры программы лежит система интерфейсов, через которые взаимодействуют все основные компоненты.

— Интерфейс `SortAlgorithm` отвечает за реализацию алгоритмов сортировки;

— Интерфейс `SortObserver` отвечает за визуализацию процесса сортировки, `DataLoader` отвечает за загрузку данных;

— Интерфейс `DataSaver` отвечает за сохранение результата.

Благодаря этому каждый модуль программы выполняет только свою задачу и может изменяться независимо от остальных частей системы.

Взаимодействие между компонентами происходит следующим образом:

					МИВУ.09.03.02 020 ПЗ	Лист
						10
Изм.	Лист	№ докум.	Подпись	Дата		

пользователь выбирает файл и тип сортировки, после чего загрузчик данных считывает информацию и передаёт массив алгоритму сортировки. Алгоритм выполняет сортировку массива в оперативной памяти и через интерфейс SortObserver отправляет обновления в модуль визуализации. Визуализатор отображает текущее состояние массива, но не влияет на сам процесс сортировки. После завершения работы отсортированный массив передаётся модулю сохранения данных, который записывает результат в файл.

Каждый интерфейс имеет собственные реализации в виде отдельных классов. Интерфейс SortAlgorithm реализуют классы BubbleSort, QuickSort и SelectionSort. Интерфейс SortObserver реализуют классы ConsoleVisualizer, SwingVisualizer и JavaFxApp. Интерфейс DataLoader реализуют классы TxtDataLoader, CsvDataLoader и JsonDataLoader, а интерфейс DataSaver — соответствующие классы для сохранения данных в нужный формат. Для создания всех этих объектов используются фабрики, которые позволяют программе выбирать нужную реализацию автоматически в зависимости от параметров запуска.

В результате была разработана модульная программа с разделением ответственности между компонентами. Сортировка выполняется независимо от визуализации, данные обрабатываются в оперативной памяти, а все основные подсистемы взаимодействуют через интерфейсы. Использование паттернов проектирования Factory и Observer позволило сделать архитектуру гибкой, расширяемой и удобной для дальнейшего развития.

В результате была разработана модульная программа с разделением ответственности между компонентами. Сортировка выполняется независимо от визуализации, данные обрабатываются в оперативной памяти, а все основные подсистемы взаимодействуют через интерфейсы. Использование паттернов проектирования Factory и Observer позволило сделать архитектуру гибкой, расширяемой и удобной для дальнейшего развития.

3. Разработка приложения

Разработка приложения выполнялась поэтапно с постепенным усложнением архитектуры и расширением функциональности программы. Основной целью разработки являлось создание универсальной системы визуализации алгоритмов сортировки, поддерживающей различные способы отображения данных, несколько форматов файлов и возможность дальнейшего расширения без изменения основной логики программы. В процессе реализации особое внимание уделялось разделению ответственности между модулями, независимости компонентов и использованию объектно-ориентированного подхода.

На начальном этапе разработки была реализована базовая структура проекта. Сначала был создан главный модуль программы, отвечающий за запуск приложения и обработку параметров командной строки. Основная задача данного модуля заключалась в получении пользовательских параметров: пути к входному файлу, выбора алгоритма сортировки, выбора визуализации и имени выходного файла. После получения параметров программа передаёт управление соответствующим фабрикам для создания необходимых объектов.

После реализации точки входа была разработана подсистема алгоритмов сортировки. Основой данной подсистемы стал интерфейс `SortAlgorithm`, определяющий единый метод сортировки массива. Интерфейс `SortAlgorithm` представлен на рисунке 1.

```
package org.example.algorithm;  
  
public interface SortAlgorithm {  
    void sort(int[] array, SortObserver observer);  
}
```

Рисунок 1 – Интерфейс `SortAlgorithm`

Использование интерфейса позволило реализовать единый подход ко всем алгоритмам и обеспечить возможность их взаимозаменяемости. На основе

					МИВУ.09.03.02 020 ПЗ	Лист
						12
Изм.	Лист	№ докум.	Подпись	Дата		

данного интерфейса были реализованы классы BubbleSort, QuickSort и SelectionSort.

Класс BubbleSort реализует алгоритм пузырьковой сортировки, при котором элементы массива последовательно сравниваются и меняются местами. После каждой перестановки вызывается обновление визуализации через интерфейс наблюдателя. Особенностью реализации стало использование передачи копии массива в визуализатор, что предотвращает проблемы с параллельным изменением данных во время отображения. Класс BubbleSort представлен на рисунке 2.

```
public class BubbleSort implements SortAlgorithm { 1 usage 2 admin +1

    @Override 4 usages 2 admin +1
    public void sort(int[] array, SortObserver observer) {

        int n = array.length;

        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {

                observer.update(array.clone(), j, index2: j + 1);

                if (array[j] > array[j + 1]) {
                    int temp = array[j];
                    array[j] = array[j + 1];
                    array[j + 1] = temp;

                    observer.update(array.clone(), j, index2: j + 1);
                }
            }
        }
    }
}
```

Рисунок 2 – класс BubbleSort

Класс QuickSort реализует алгоритм быстрой сортировки с использованием рекурсивного разделения массива. В процессе разработки особое внимание уделялось корректной передаче индексов сравниваемых элементов в визуализатор, так как алгоритм выполняет большое количество промежуточных операций. Реализация алгоритма была адаптирована под систему наблюдателей, благодаря чему визуализация обновляется после каждого обмена элементов. Класс QuickSort представлен на рисунке 3.

```

public class QuickSort implements SortAlgorithm { 1 usage  ⚙ admin +1

    @Override 4 usages  ⚙ admin
    public void sort(int[] array, SortObserver observer) { quickSort(array, low: 0, high: array.length - 1, observer); }

    private void quickSort(int[] array, int low, int high, SortObserver observer) { 3 usages  ⚙ admin

        if (low < high) {
            int pi = partition(array, low, high, observer);
            quickSort(array, low, high: pi - 1, observer);
            quickSort(array, low: pi + 1, high, observer);
        }
    }
}

```

Рисунок 3 – класс QuickSort

Класс SelectionSort реализует сортировку выбором. Во время выполнения алгоритма производится поиск минимального элемента и его последующая перестановка с текущей позицией массива. Аналогично другим алгоритмам, после каждой значимой операции вызывается метод обновления визуализации. Класс SelectionSort представлен на рисунке 4.

```

public class SelectionSort implements SortAlgorithm { 2 usages  ⚙ admin +1

    @Override 4 usages  ⚙ admin +1
    public void sort(int[] array, SortObserver observer) {

        int n = array.length;

        for (int i = 0; i < n - 1; i++) {

            int minIndex = i;

            for (int j = i + 1; j < n; j++) {
                // подсветка сравниваемых элементов
                observer.update(array.clone(), j, minIndex);

                if (array[j] < array[minIndex]) {
                    minIndex = j;
                    observer.update(array.clone(), j, minIndex);
                }
            }

            if (minIndex != i) {
                int temp = array[i];
                array[i] = array[minIndex];
                array[minIndex] = temp;

                observer.update(array.clone(), i, minIndex);
            }
        }
    }
}

```

Рисунок 4 – класс SelectionSort

После завершения реализации алгоритмов была разработана система визуализации. Основной задачей данного этапа стало создание независимого

механизма отображения, который не влияет на работу алгоритмов сортировки. Для этого был реализован интерфейс SortObserver, через который алгоритмы передают текущее состояние массива визуализатору. Интерфейс SortObserver представлен на рисунке 5.

```
public interface SortObserver {
    default void update(int[] array, int index1, int index2) {
    }
    default void update(int[] array) { update(array, index1: -1, index2: -1); }
}
```

Рисунок 5 – Интерфейс SortObserver

Первым вариантом визуализации стала консольная реализация ConsoleVisualizer. Данный класс выводит массив в текстовом формате после каждого изменения. На этапе разработки дополнительно была реализована подсветка элементов через ANSI-последовательности терминала, благодаря чему процесс сортировки стал более наглядным даже в консольном режиме. Также между обновлениями была добавлена задержка, позволяющая визуально отслеживать изменения массива. класс ConsoleVisualizer представлен на рисунке 6.

```
public class ConsoleVisualizer implements SortObserver {
    @Override
    public void update(int[] array, int i, int j) {
    }
    private void drawBars(int[] array, int i, int j) {
    }
    private void clearConsole() {
    }
}
```

Рисунок 6 – Класс ConsoleVisualizer

Следующим этапом стала реализация графической визуализации на основе библиотеки Swing. Класс SwingVisualizer отображает массив в виде набора прямоугольников, высота которых соответствует значению элементов массива. Во

время разработки возникла необходимость корректного обновления графического интерфейса без блокировки основного потока выполнения. Для этого визуализация была реализована через механизм перерисовки компонентов Swing. Дополнительно были добавлены отображение числовых значений элементов массива, цветовая подсветка сравниваемых элементов и улучшенное оформление интерфейса. Класс SwingVisualizer представлен на рисунке 7.

```

12 public class SwingVisualizer extends JFrame implements SortObserver {
13
14     private int[] data; 11 usages
15     private String outputPath; 4 usages
16
17     private final DrawPanel drawPanel = new DrawPanel(); 3 usages
18
19     private JComboBox<String> algorithmBox; 3 usages
20     private JSlider speedSlider; 6 usages
21
22     private int index1 = -1; 2 usages
23     private int index2 = -1; 2 usages
24
25 > public SwingVisualizer() {...}
36
37 > private void initUI() {...}
14
15 @Override 9 usages darya
16 > public void update(int[] array, int i, int j) {...}
27
28 > class DrawPanel extends JPanel {...}
63 }

```

Рисунок 7 – Класс SwingVisualizer

Наиболее функциональной частью проекта стала реализация визуализации на основе JavaFX. Класс JavaFxApp был разработан как отдельное графическое приложение с современным интерфейсом пользователя. В процессе разработки были реализованы выбор входного и выходного файла через графический интерфейс, настройка скорости анимации при помощи ползунка, выбор алгоритма сортировки и отображение процесса сортировки в реальном времени. Для визуализации использовались графические элементы Rectangle, формирующие столбчатую диаграмму массива. Особое внимание уделялось синхронизации

потоков, так как сортировка выполняется в отдельном потоке, а обновление интерфейса должно происходить только через Platform.runLater(). Класс JavaFxApp представлен на рисунке 8.

```
public class JavaFxApp extends Application implements SortObserver {  @darya

    private int[] data; 5 usages
    private String outputPath; 6 usages

    private Pane drawPane = new Pane(); 9 usages
    private List<Rectangle> bars = new ArrayList<>(); 2 usages

    private Slider speedSlider = new Slider(v: 1, v1: 100, v2: 50); 4 usages

    @Override @darya
    public void start(Stage stage) {...}

    private void drawArray(int[] array, int index1, int index2) { 2 usages @darya
        Platform.runLater(() -> {...});
    }

    @Override 9 usages @darya
    public void update(int[] array, int i, int j) {...}

    private void showAlert(String text) {...}
}
```

Рисунок 8 – Класс JavaFxApp

После реализации визуализации была разработана подсистема работы с файлами. Для поддержки нескольких форматов данных были созданы интерфейсы DataLoader и DataSaver. Каждый загрузчик реализует собственный способ чтения данных из файла и преобразования их в массив. Интерфейсы DataLoader и DataSaver представлены на рисунках 9-10.

```
public interface DataLoader { 7
    int[] load(String filePath);
}
```

Рисунок 9 – Интерфейс DataLoader

```
public interface DataSaver { 7 usages 3 implement
    void save(int[] array, String filePath);
}
```

Рисунок 10 – Интерфейс DataSaver

Класс TxtDataLoader выполняет чтение чисел из текстового файла. В процессе разработки была реализована обработка ошибок чтения и проверка корректности входных данных. Класс TxtDataLoader представлен на рисунке 11.

```
public class TxtDataLoader implements DataLoader { 4 usages admin *

    @Override 6 usages admin *
    public int[] load(String filePath) {

        try {
            String content = new String(Files.readAllBytes(Paths.get(filePath)));
            content = content.replace(target: "\uFEFF", replacement: "");
            content = content.trim();
            String[] numbers = content.split(regex: ",");
            List<Integer> list = new ArrayList<>();

            for (String num : numbers) {
                if (!num.trim().isEmpty()) {
                    list.add(Integer.parseInt(num.trim()));
                }
            }

            return list.stream().mapToInt(Integer::intValue).toArray();

        } catch (IOException e) {
            throw new RuntimeException("Ошибка чтения TXT файла: " + filePath);
        }
    }
}
```

Рисунок 11 – Класс TxtDataLoader

Класс CsvDataLoader предназначен для чтения CSV-файлов, в которых элементы разделяются запятыми. Класс CsvDataLoader представлен на рисунке 12.

```

public class CsvDataLoader implements DataLoader { 4 usages  admin

    @Override 6 usages  admin
    public int[] load(String filePath) {

        try {
            String content = new String(Files.readAllBytes(Paths.get(filePath)));

            // Убираем пробелы и переносы строк
            content = content.replaceAll(regex: "\\s", replacement: "");

            String[] numbers = content.split(regex: ",");

            int[] array = new int[numbers.length];

            for (int i = 0; i < numbers.length; i++) {
                array[i] = Integer.parseInt(numbers[i]);
            }

            return array;
        } catch (IOException e) {
            throw new RuntimeException("Ошибка чтения CSV файла: " + filePath);
        }
    }
}

```

Рисунок 12 – Класс CsvDataLoader

Класс JsonDataLoader выполняет чтение JSON-массива и преобразование его в массив целых чисел. Класс JsonDataLoader представлен на рисунке 13.

```

public class CsvDataLoader implements DataLoader { 4 usages  admin *

    @Override 6 usages  admin *
    public int[] load(String filePath) {

        try {
            String content = new String(Files.readAllBytes(Paths.get(filePath)));
            content = content.replaceAll(regex: "\\s", replacement: "");
            String[] numbers = content.split(regex: ",");
            int[] array = new int[numbers.length];

            for (int i = 0; i < numbers.length; i++) {
                array[i] = Integer.parseInt(numbers[i]);
            }

            return array;
        } catch (IOException e) {
            throw new RuntimeException("Ошибка чтения CSV файла: " + filePath);
        }
    }
}

```

Рисунок 13 – Класс JsonDataLoader

Аналогичным образом были реализованы классы сохранения данных в соответствующие форматы файлов.

Для уменьшения связанности компонентов и повышения гибкости архитектуры в проекте была реализована система фабрик. Необходимость использования фабрик возникла на этапе проектирования программы, когда стало понятно, что приложение должно поддерживать несколько алгоритмов сортировки, различные способы визуализации и несколько форматов файлов. При прямом создании объектов в основном коде программы пришлось бы использовать большое количество условных операторов if/else или switch, что значительно усложнило бы поддержку и расширение проекта.

Для решения данной проблемы были реализованы специальные классы-фабрики, задачей которых является создание нужных объектов в зависимости от параметров, выбранных пользователем. Такой подход позволил вынести всю логику создания объектов в отдельные модули и сделать основной код программы более универсальным.

Первой была реализована фабрика алгоритмов сортировки `AlgorithmFactory`. Данный класс отвечает за создание объекта нужного алгоритма на основе строки, переданной пользователем. Например, если пользователь указывает параметр "quick", фабрика создаёт объект класса `QuickSort`, а при выборе "bubble" создаётся объект `BubbleSort`. Внутри фабрики реализован метод `create()`, который анализирует входной параметр и возвращает соответствующую реализацию интерфейса `SortAlgorithm`. Благодаря этому основной модуль программы не зависит от конкретных классов алгоритмов и работает только через интерфейс. При добавлении нового алгоритма сортировки достаточно создать новый класс и зарегистрировать его в фабрике без изменения остальной логики программы.

Аналогичным образом была реализована фабрика визуализации `VisualizerFactory`. Она отвечает за создание нужного варианта отображения процесса сортировки. В зависимости от параметров запуска фабрика может создать объект `ConsoleVisualizer`, `SwingVisualizer` или подключить визуализацию на основе `JavaFX`. Использование отдельной фабрики визуализации позволило

					МИВУ.09.03.02 020 ПЗ	Лист
						20
Изм.	Лист	№ докум.	Подпись	Дата		

полностью отделить систему отображения от алгоритмов сортировки. Алгоритмы не знают, какой именно визуализатор используется, а работают только через интерфейс SortObserver. Это обеспечило независимость логики сортировки от конкретной библиотеки интерфейса.

Следующим этапом была разработка фабрики загрузчиков данных DataLoaderFactory. Поскольку программа поддерживает несколько форматов файлов, было необходимо автоматически определять нужный способ чтения данных. Фабрика анализирует расширение файла и создаёт соответствующий загрузчик: TxtDataLoader, CsvDataLoader или JsonDataLoader. Благодаря этому основной код программы не содержит логики проверки форматов файлов и работает одинаково независимо от типа входных данных.

По аналогичному принципу была реализована фабрика сохранения данных DataSaverFactory. Она отвечает за выбор класса, сохраняющего результат сортировки в нужный формат файла. В зависимости от расширения выходного файла создаётся соответствующий объект сохранения данных. Такой подход позволил унифицировать процесс записи данных и обеспечить одинаковый механизм работы со всеми форматами файлов.

Использование фабрик позволило реализовать в проекте паттерн Factory Method, основной задачей которого является инкапсуляция создания объектов. Благодаря этому удалось существенно уменьшить связанность компонентов программы. Основной модуль больше не зависит от конкретных реализаций алгоритмов, визуализаторов или загрузчиков файлов и взаимодействует только через интерфейсы. Это значительно упростило архитектуру программы, повысило её расширяемость и облегчило сопровождение проекта.

Дополнительным преимуществом использования фабрик стало упрощение масштабирования программы. Например, для добавления нового алгоритма сортировки требуется только:

- Создать новый класс, реализующий интерфейс SortAlgorithm;
- Добавить его создание в AlgorithmFactory

Аналогичным образом можно добавлять новые визуализаторы или форматы

файлов без изменения основной структуры приложения. Такой подход соответствует принципу открытости и закрытости из SOLID, согласно которому система должна быть открыта для расширения, но закрыта для изменения существующего кода.

Важной особенностью разработки стало обеспечение независимости сортировки от визуализации. Алгоритмы работают исключительно с массивом данных, расположенным в оперативной памяти, и не содержат логики отображения. Визуализаторы получают только копии текущего состояния массива через интерфейс наблюдателя. Благодаря такому подходу одна и та же реализация алгоритма может использоваться одновременно с несколькими вариантами отображения без изменения кода сортировки.

Дополнительно в процессе разработки была реализована система автоматической сборки проекта с использованием Maven. В файл `pom.xml` были добавлены зависимости JavaFX, плагины для запуска приложения, а также инструменты тестирования и анализа кода. Для автоматизации проверки проекта был настроен GitHub Actions, выполняющий сборку и тестирование программы при загрузке изменений в репозиторий.

На завершающем этапе были разработаны модульные тесты с использованием библиотеки JUnit. Тестирование охватывает работу алгоритмов сортировки и загрузчиков файлов различных форматов. Для анализа покрытия кода был подключён инструмент JaCoCo, формирующий отчёт о проценте протестированных участков программы. Также был настроен статический анализ кода с использованием Checkstyle.

В результате разработки была создана модульная расширяемая система визуализации алгоритмов сортировки, поддерживающая несколько алгоритмов, различные форматы файлов и несколько вариантов визуализации. Архитектура программы основана на интерфейсах и паттернах проектирования, что обеспечивает независимость компонентов, упрощает сопровождение проекта и позволяет легко добавлять новый функционал без изменения существующей логики системы.

					МИВУ.09.03.02 020 ПЗ	Лист
						22
Изм.	Лист	№ докум.	Подпись	Дата		

4. Тестирование

Одним из важнейших этапов создания программного продукта является его тестирование и отладка. Тестирование позволяет выявить скрытые и явные недостатки программы, либо убедиться в ее пригодности для применения.

Целью тестирования является проверка работоспособности программы, то есть правильности выполнения всех функций, описанных выше.

Таблица 1 – Тестирование

№	Действие	Результат
1	2	3
1.	Запуск функции help	Успех, корректное отображение информации
2.	Попытка передать несуществующий параметр алгоритма сортировки	Вывод сообщения «Неизвестный алгоритм: wrong»
3.	Попытка запуска программы с консольной визуализацией	Успех, сортировка массива и сохранение результата в указанный файл
4.	Попытка запуска программы с визуализацией через библиотеку Swing	Успех, сортировка массива
5.	Попытка сохранения результата с визуализацией через библиотеку Swing	Успех, сохранение файла
6.	Попытка запуска программы с визуализацией через библиотеку JavaFX	Успех
7.	Попытка запуска программы с выбором алгоритма сортировки и файлами для загрузки и сохранения результата	Успех, сортировка массива и сохранение файла
8.	Попытка провести maven-тесты	Успех, выполнение всех тестов
9.	Попытка провести статический анализ	Успех

Вывод сообщений об ошибках и другие сообщения представлены на рисунках 14-24.


```
[admin@localhost smakotina_kyrsovaya_java]$ java -jar target/smakotina_kyrsovaya_java-1.0-SNAPSHOT.jar --help
Использование:
java -jar app.jar <input> <algorithm> <visualization> <output>

Пример:
java -jar app.jar forjson.json quick console result.json

Поддерживаемые форматы входа: .json .csv .txt
Поддерживаемые форматы выхода: .json .csv .txt
Алгоритмы: quick, bubble, selection
```

Рисунок 14 – Вывод результата теста №1

```
[admin@localhost smakotina_kyrsovaya_java]$ java -jar target/smakotina_kyrsovaya_java-1.0-SNAPSHOT.jar forjson.json wrong console result.json
Ошибка: Неизвестный алгоритм: wrong
```

Рисунок 15 – Вывод результата теста №2



Рисунок 16 – Вывод результата теста №3

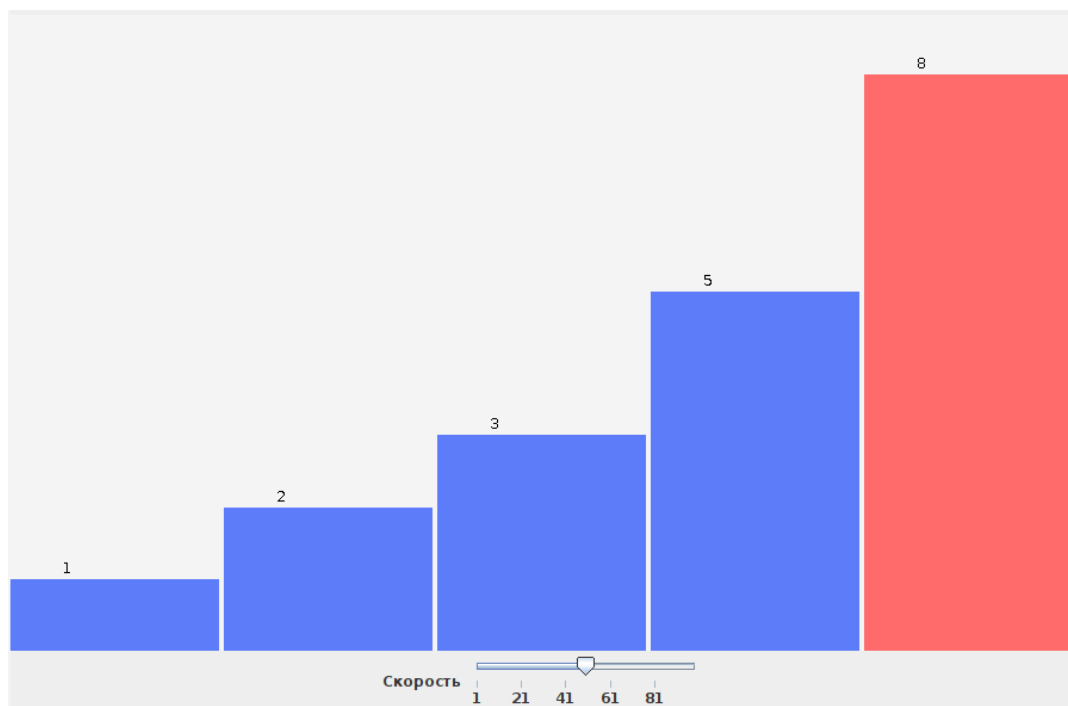


Рисунок 17 – Вывод результата теста №4

```
[admin@localhost smakotina_kyrsovaya_java]$ java -jar target/smakotina_kyrsovaya_java-1.0-SNAPSHOT.jar forjson.json quick swing output.json
Результат сохранён в файл: output.json
Сортировка завершена.
█
```

Рисунок 18 – Вывод результата теста №5

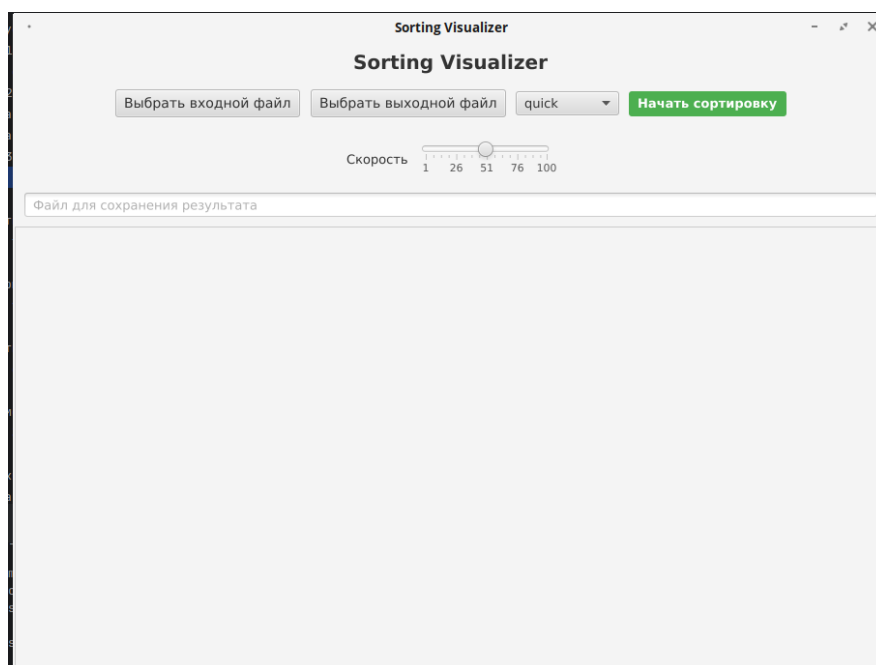


Рисунок 19 – Вывод результата теста №6

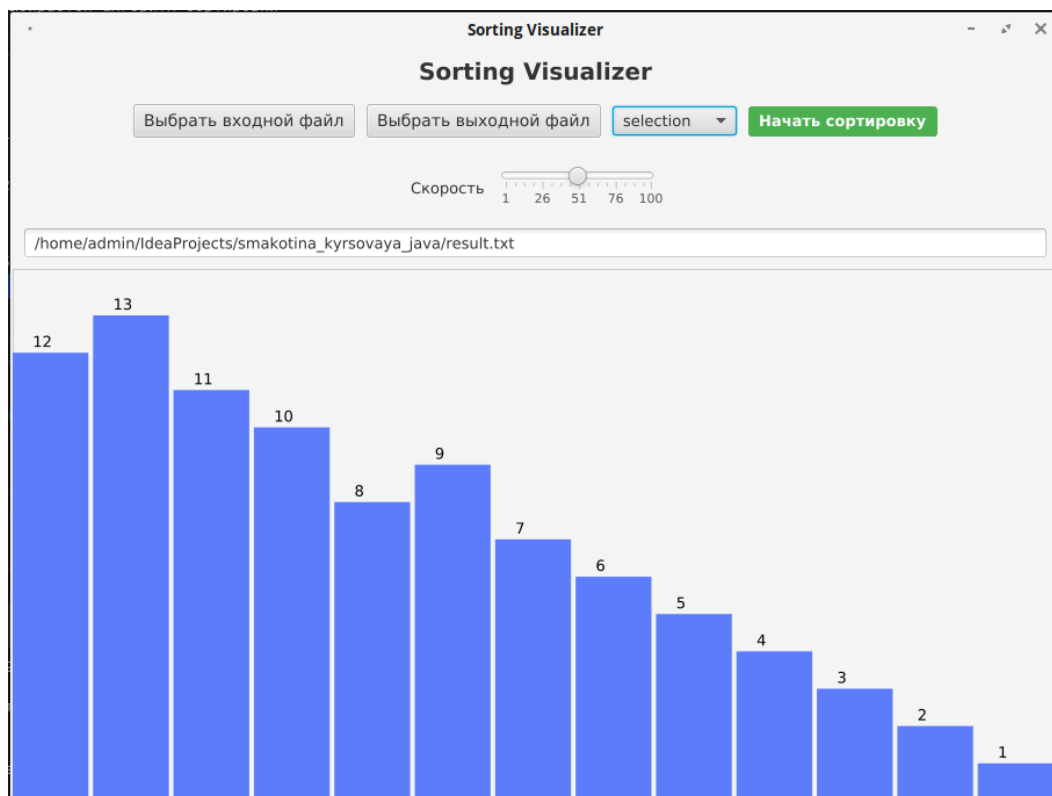


Рисунок 20 – Вывод результата теста №7

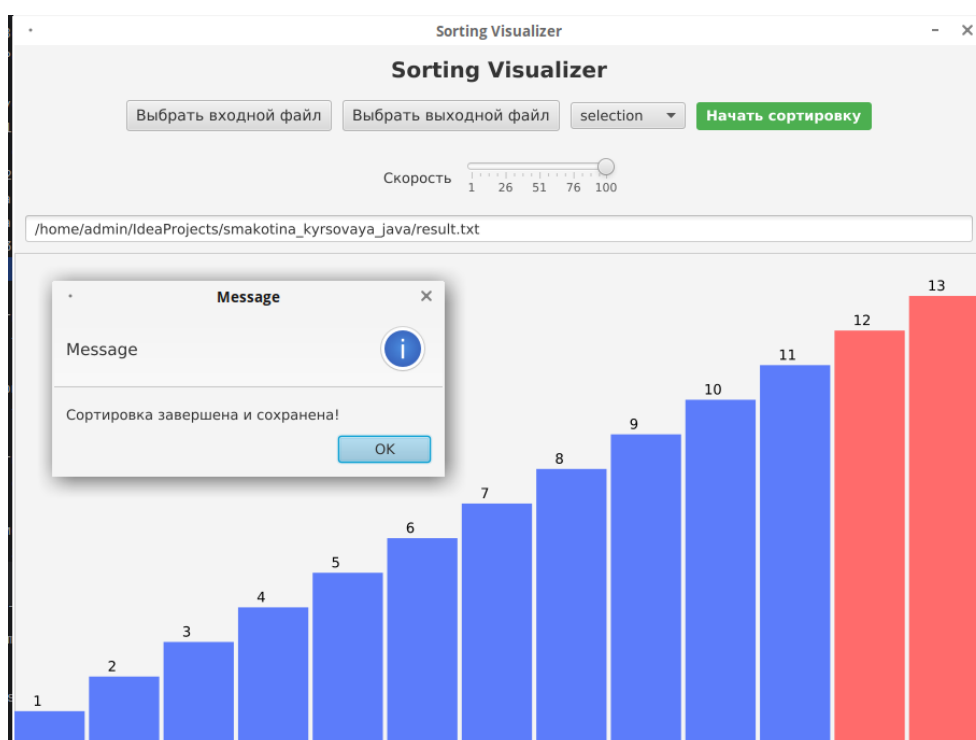


Рисунок 21 – Вывод результата теста №7

Результат сохранён в файл: /home/admin/IdeaProjects/smakotina_kyrsovaya_java/result.txt

Рисунок 22 – Вывод результата теста №7

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running org.example.algorithm.SortTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.097 s -- in org.example.algorithm.SortTest
[INFO] Results:
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco:0.8.11:report (report) @ smakotina_kyrsovaya_java ---
[INFO] Loading execution data file /home/admin/IdeaProjects/smakotina_kyrsovaya_java/target/jacoco.exec
[INFO] Analyzed bundle 'smakotina_kyrsovaya_java' with 26 classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.393 s
[INFO] Finished at: 2026-05-13T10:02:00+03:00
[INFO] -----
```

Рисунок 23 – Вывод результата теста №8

```
[admin@localhost smakotina_kyrsovaya_java]$ mvn checkstyle:check
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.example:smakotina_kyrsovaya_java >-----
[INFO] Building smakotina_kyrsovaya_java 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- checkstyle:3.3.1:check (default-cli) @ smakotina_kyrsovaya_java ---
[INFO] You have 0 Checkstyle violations.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.227 s
[INFO] Finished at: 2026-05-13T10:05:56+03:00
[INFO] -----
```

Рисунок 24 – Вывод результата теста №9

Заключение

В ходе выполнения курсовой работы была разработана программная система визуализации алгоритмов сортировки на языке Java. Основной целью проекта являлось создание расширяемого приложения, способного выполнять сортировку данных, загруженных из файлов различных форматов, и отображать процесс сортировки несколькими способами визуализации.

В процессе разработки была спроектирована модульная архитектура программы, включающая подсистемы алгоритмов сортировки, визуализации, загрузки и сохранения данных, а также систему фабрик для создания объектов. Такой подход позволил разделить ответственность между компонентами программы и обеспечить их независимость друг от друга.

В рамках работы были реализованы алгоритмы пузырьковой сортировки, быстрой сортировки и сортировки выбором. Для взаимодействия компонентов использовались интерфейсы SortAlgorithm, SortObserver, DataLoader и DataSaver. Сортировка выполняется в оперативной памяти, а визуализация только отображает текущее состояние массива, не влияя на работу алгоритмов.

Для отображения процесса сортировки были реализованы три варианта визуализации: консольная визуализация, графическая визуализация на основе Swing и JavaFX. Также была добавлена поддержка форматов TXT, CSV и JSON для загрузки и сохранения данных.

При разработке были использованы паттерны проектирования Factory, Observer и Strategy, что обеспечило гибкость архитектуры и возможность дальнейшего расширения программы. Дополнительно были настроены Maven, GitHub Actions, модульное тестирование с использованием JUnit, а также инструменты анализа кода JaCoCo и Checkstyle.

В результате выполнения курсовой работы была создана полнофункциональная система визуализации алгоритмов сортировки, соответствующая поставленным требованиям и демонстрирующая применение

					МИВУ.09.03.02 020 ПЗ	Лист
						28
Изм.	Лист	№ докум.	Подпись	Дата		

принципов объектно-ориентированного программирования и современных средств разработки программного обеспечения.

					МИВУ.09.03.02 020 ПЗ	Лист
						29
Изм.	Лист	№ докум.	Подпись	Дата		

Библиография

1. Системы управления базами данных [Электронный ресурс]: лабораторный практикум/ – Электрон текстовые данные. – Ставрополь: Северо-Кавказский федеральный университет, 2017. – 148 с. – Режим доступа: <http://www.iprbookshop.ru/75595.html>. – ЭБС «IPRbooks»
2. Лазицкас Е.А. Базы данных и системы управления базами данных [Электронный ресурс]: учебное пособие/ Лазицкас Е.А., Загумённикова И.Н., Гилевский П.Г. – Электрон. текстовые данные. – Минск: Республиканский институт профессионального образования (РИПО), 2018. – 268 с. – Режим доступа: <http://www.iprbookshop.ru/93382.html>. – ЭБС «IPRbooks»
3. Ермаков, А. В. Разработка приложений на языке JAVA : учебное пособие / А. В. Ермаков, М. С. Королёв, А. К. Кузьмин. — Саратов : Саратовский государственный технический университет имени Ю.А. Гагарина, ЭБС АСВ, 2024. — 128 с. — ISBN 978-5-7433-3635-7. — Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. — URL: <https://www.iprbookshop.ru/150088.html>
4. Java. Библиотека профессионала. Том 1. Основы [Электронный ресурс]: / К. С. Хорстманн, Г. Корнелл. — Электрон. текстовые данные. — М.: Вильямс, 2019. — 832 с. — Режим доступа: <https://www.litres.ru/key-horstmann/java-biblioteka-professional-a-tom-1-osnovy/>

